Pr 1: Perform encryption and decryption using Caesar cipher Algorithm.

Code :

```
public class CaesarCipher {

  // Method to encrypt text
  public static String encrypt(String text, int key) {
    StringBuilder result = new StringBuilder();

    // Normalize key (handles keys > 26)
    int shift = key % 26;

    for (int i = 0; i < text.length(); i++) {
      char ch = text.charAt(i);

      // For uppercase letters
      if (Character.isUpperCase(ch)) {
        char encryptedChar = (char) ('A' + (ch - 'A' + shift) % 26);
        result.append(encryptedChar);
      }
      // For lowercase letters
      else if (Character.isLowerCase(ch)) {
        char encryptedChar = (char) ('a' + (ch - 'a' + shift) % 26);
        result.append(encryptedChar);
      }
      // For spaces or symbols (unchanged)
      else {
        result.append(ch);
      }
    }

    return result.toString();
```

```
    }

    // Method to decrypt text
    public static String decrypt(String text, int key) {
        int shift = 26 - (key % 26); // reverse shift
        return encrypt(text, shift);
    }

    public static void main(String[] args) {

        String originalText = "Hello World";
        int key = 3;

        System.out.println("Original Text: " + originalText);

        String encrypted = encrypt(originalText, key);
        System.out.println("Encrypted Text: " + encrypted);

        String decrypted = decrypt(encrypted, key);
        System.out.println("Decrypted Text: " + decrypted);
    }
}
```

Pr 2: Perform encryption and decryption using Play Fair Cipher.

Code:

```
import java.util.*;

public class PlayfairCipher {

  private char[][] matrix = new char[5][5];
  private String key;

  // Constructor
  public PlayfairCipher(String key) {
    this.key = key;
    generateMatrix();
  }

  // Generate 5x5 key matrix
  private void generateMatrix() {
    StringBuilder unique = new StringBuilder();

    key = key.toUpperCase().replaceAll("[^A-Z]", "").replace('J', 'I');

    for (char c : key.toCharArray()) {
      if (unique.indexOf(String.valueOf(c)) == -1) {
        unique.append(c);
      }
    }

    String alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ";

    for (char c : alphabet.toCharArray()) {
      if (unique.indexOf(String.valueOf(c)) == -1) {
```

```
            unique.append(c);
        }
    }

    // Fill matrix
    for (int i = 0; i < 25; i++) {
        matrix[i / 5][i % 5] = unique.charAt(i);
    }
}

// Prepare plaintext (remove spaces, handle duplicates)
private String prepareText(String text) {
    text = text.toUpperCase().replaceAll("[^A-Z]", "").replace('J', 'I');
    StringBuilder result = new StringBuilder();

    for (int i = 0; i < text.length(); i++) {
        result.append(text.charAt(i));

        if (i < text.length() - 1 && text.charAt(i) == text.charAt(i + 1)) {
            result.append('X');
        }
    }

    if (result.length() % 2 != 0) {
        result.append('X');
    }

    return result.toString();
}

// Get position of character in matrix
```

```
private int[] getPosition(char c) {
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++) {
            if (matrix[i][j] == c) {
                return new int[]{i, j};
            }
        }
    }
    return null;
}

// Encrypt
public String encrypt(String text) {
    text = prepareText(text);
    StringBuilder cipher = new StringBuilder();

    for (int i = 0; i < text.length(); i += 2) {
        char a = text.charAt(i);
        char b = text.charAt(i + 1);

        int[] posA = getPosition(a);
        int[] posB = getPosition(b);

        int r1 = posA[0], c1 = posA[1];
        int r2 = posB[0], c2 = posB[1];

        // Same row
        if (r1 == r2) {
            cipher.append(matrix[r1][(c1 + 1) % 5]);
            cipher.append(matrix[r2][(c2 + 1) % 5]);
        }
```

```
        // Same column
        else if (c1 == c2) {
            cipher.append(matrix[(r1 + 1) % 5][c1]);
            cipher.append(matrix[(r2 + 1) % 5][c2]);
        }
        // Rectangle
        else {
            cipher.append(matrix[r1][c2]);
            cipher.append(matrix[r2][c1]);
        }
    }

    return cipher.toString();
}

// Decrypt
public String decrypt(String text) {
    StringBuilder plain = new StringBuilder();

    for (int i = 0; i < text.length(); i += 2) {
        char a = text.charAt(i);
        char b = text.charAt(i + 1);

        int[] posA = getPosition(a);
        int[] posB = getPosition(b);

        int r1 = posA[0], c1 = posA[1];
        int r2 = posB[0], c2 = posB[1];

        // Same row
        if (r1 == r2) {
```

```
            plain.append(matrix[r1][(c1 + 4) % 5]);
            plain.append(matrix[r2][(c2 + 4) % 5]);
        }
        // Same column
        else if (c1 == c2) {
            plain.append(matrix[(r1 + 4) % 5][c1]);
            plain.append(matrix[(r2 + 4) % 5][c2]);
        }
        // Rectangle
        else {
            plain.append(matrix[r1][c2]);
            plain.append(matrix[r2][c1]);
        }
    }

    return plain.toString();
}

// Display matrix
public void printMatrix() {
    System.out.println("\nKey Matrix:");
    for (char[] row : matrix) {
        for (char c : row) {
            System.out.print(c + " ");
        }
        System.out.println();
    }
}

public static void main(String[] args) {
    String key = "EXAMPLE";
```

```
        String message = "Hide the gold";

        PlayfairCipher cipher = new PlayfairCipher(key);

        System.out.println("Original Message: " + message);

        String encrypted = cipher.encrypt(message);
        System.out.println("Encrypted Message: " + encrypted);

        String decrypted = cipher.decrypt(encrypted);
        System.out.println("Decrypted Message: " + decrypted);

        cipher.printMatrix();
    }
}
```

Pr 3: Perform encryption and decryption using Rail Fence Technique. (Row Transposition Techniques)

Code :

```
public class RailFenceCipher {


  // Clean text (remove spaces, convert to uppercase)
  private static String cleanText(String text) {
    return text.toUpperCase().replaceAll("[^A-Z]", "");
  }


  // Encryption
  public static String encrypt(String text, int rails) {
    String clean = cleanText(text);


    if (rails <= 1) return clean;


    StringBuilder[] fence = new StringBuilder[rails];
    for (int i = 0; i < rails; i++) {
      fence[i] = new StringBuilder();
    }


    int row = 0;
    boolean down = true;


    for (char c : clean.toCharArray()) {
      fence[row].append(c);


      if (row == 0) down = true;
      else if (row == rails - 1) down = false;


      row += down ? 1 : -1;
```

```
    }

    // Combine rows
    StringBuilder result = new StringBuilder();
    for (StringBuilder sb : fence) {
        result.append(sb);
    }

    return result.toString();
}

// Decryption
public static String decrypt(String cipher, int rails) {

    if (rails <= 1) return cipher;

    int len = cipher.length();
    char[][] fence = new char[rails][len];

    // Mark zigzag path
    int row = 0;
    boolean down = true;

    for (int i = 0; i < len; i++) {
        fence[row][i] = '*';

        if (row == 0) down = true;
        else if (row == rails - 1) down = false;

        row += down ? 1 : -1;
    }
```

```
    // Fill characters row-wise
    int index = 0;
    for (int i = 0; i < rails; i++) {
        for (int j = 0; j < len; j++) {
            if (fence[i][j] == '*' && index < len) {
                fence[i][j] = cipher.charAt(index++);
            }
        }
    }

    // Read zigzag
    StringBuilder result = new StringBuilder();
    row = 0;
    down = true;

    for (int i = 0; i < len; i++) {
        result.append(fence[row][i]);

        if (row == 0) down = true;
        else if (row == rails - 1) down = false;

        row += down ? 1 : -1;
    }

    return result.toString();
}

public static void main(String[] args) {

    String message = "WELCOME TO THE NEXT LEVEL";
```

```
        int rails = 3;

        System.out.println("Original Message: " + message);

        String encrypted = encrypt(message, rails);
        System.out.println("Encrypted Message: " + encrypted);

        String decrypted = decrypt(encrypted, rails);
        System.out.println("Decrypted Message: " + decrypted);
    }
}
```

Pr4: Perform encryption and decryption using (Columnar Transposition Techniques)

Code: import java.util.*;

```
public class ColumnarCipher {

  // Clean text (uppercase, remove spaces)
  private static String cleanText(String text) {
    return text.toUpperCase().replaceAll("[^A-Z]", "");
  }

  // Get column order based on key
  private static int[] getOrder(String key) {
    key = key.toUpperCase();
    int len = key.length();

    Character[] keyArr = new Character[len];
    for (int i = 0; i < len; i++) keyArr[i] = key.charAt(i);

    Character[] sorted = keyArr.clone();
    Arrays.sort(sorted);

    int[] order = new int[len];

    for (int i = 0; i < len; i++) {
      for (int j = 0; j < len; j++) {
        if (keyArr[i] == sorted[j]) {
          order[i] = j;
          sorted[j] = null; // avoid duplicate matching
          break;
        }
      }
```

```
    }
    return order;
}

// Encryption
public static String encrypt(String text, String key) {
    text = cleanText(text);
    int cols = key.length();
    int rows = (int) Math.ceil((double) text.length() / cols);

    char[][] matrix = new char[rows][cols];

    // Fill matrix row-wise
    int index = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (index < text.length())
                matrix[i][j] = text.charAt(index++);
            else
                matrix[i][j] = 'X'; // padding
        }
    }

    int[] order = getOrder(key);

    StringBuilder cipher = new StringBuilder();

    // Read column-wise based on order
    for (int k = 0; k < cols; k++) {
        for (int j = 0; j < cols; j++) {
            if (order[j] == k) {
```

```
            for (int i = 0; i < rows; i++) {
                cipher.append(matrix[i][j]);
            }
        }
    }
  }

  return cipher.toString();
}

// Decryption
public static String decrypt(String cipher, String key) {
  cipher = cleanText(cipher);
  int cols = key.length();
  int rows = (int) Math.ceil((double) cipher.length() / cols);

  char[][] matrix = new char[rows][cols];

  int[] order = getOrder(key);

  int index = 0;

  // Fill column-wise
  for (int k = 0; k < cols; k++) {
    for (int j = 0; j < cols; j++) {
      if (order[j] == k) {
        for (int i = 0; i < rows; i++) {
          if (index < cipher.length())
            matrix[i][j] = cipher.charAt(index++);
        }
      }
```

```
        }
    }

    // Read row-wise
    StringBuilder plain = new StringBuilder();
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            plain.append(matrix[i][j]);
        }
    }

    return plain.toString();
  }

  public static void main(String[] args) {

    String message = "TRANSPORTATION";
    String key = "HACK";

    System.out.println("Original Message: " + message);

    String encrypted = encrypt(message, key);
    System.out.println("Encrypted Message: " + encrypted);

    String decrypted = decrypt(encrypted, key);
    System.out.println("Decrypted Message: " + decrypted);
  }
}
```

Pr 5: Perform encryption and decryption using One Time Pad Algorithm.

Code:

```
public class OneTimePadCipher {

  // Clean text (uppercase, remove spaces)
  private static String cleanText(String text) {
    return text.toUpperCase().replaceAll("[^A-Z]", "");
  }

  // Validate key length
  private static void validate(String text, String key) {
    if (text.length() != key.length()) {
      throw new IllegalArgumentException(
        "Key length must be same as message length"
      );
    }
  }

  // Encryption
  public static String encrypt(String message, String key) {
    message = cleanText(message);
    key = cleanText(key);

    validate(message, key);

    StringBuilder cipher = new StringBuilder();

    for (int i = 0; i < message.length(); i++) {
      int m = message.charAt(i) - 'A';
      int k = key.charAt(i) - 'A';
```

```
        int encrypted = (m + k) % 26;

        cipher.append((char) ('A' + encrypted));
    }

    return cipher.toString();
}

// Decryption
public static String decrypt(String cipher, String key) {
    cipher = cleanText(cipher);
    key = cleanText(key);

    validate(cipher, key);

    StringBuilder plain = new StringBuilder();

    for (int i = 0; i < cipher.length(); i++) {
        int c = cipher.charAt(i) - 'A';
        int k = key.charAt(i) - 'A';

        int decrypted = (c - k + 26) % 26;

        plain.append((char) ('A' + decrypted));
    }

    return plain.toString();
}

public static void main(String[] args) {
```

```
        String message = "HELLO";
        String key = "XMCKL"; // same length as message

        System.out.println("Original Message: " + message);
        System.out.println("Key: " + key);

        String encrypted = encrypt(message, key);
        System.out.println("Encrypted Message: " + encrypted);

        String decrypted = decrypt(encrypted, key);
        System.out.println("Decrypted Message: " + decrypted);
    }
}
```

Pr 6: Write a program to Develop a Secure System by Applying RSA Cryptography Algorithms.

Code: import java.math.BigInteger;

```
import java.security.SecureRandom;

public class RSAExample {

  private BigInteger p, q, n, phi, e, d;
  private int bitLength = 512; // smaller for demo

  // Constructor (generate keys)
  public RSAExample() {
    SecureRandom random = new SecureRandom();

    // Step 1: choose two prime numbers
    p = BigInteger.probablePrime(bitLength / 2, random);
    q = BigInteger.probablePrime(bitLength / 2, random);

    // Step 2: calculate n and phi
    n = p.multiply(q);
    phi = (p.subtract(BigInteger.ONE)).multiply(q.subtract(BigInteger.ONE));

    // Step 3: choose e (public key)
    e = BigInteger.valueOf(65537);

    // Step 4: calculate d (private key)
    d = e.modInverse(phi);
  }

  // Encryption
  public BigInteger encrypt(BigInteger message) {
    return message.modPow(e, n);
```

```
    }

    // Decryption
    public BigInteger decrypt(BigInteger cipher) {
        return cipher.modPow(d, n);
    }

    public static void main(String[] args) {

        RSAExample rsa = new RSAExample();

        String message = "HELLO";

        // Convert string to number
        BigInteger msg = new BigInteger(message.getBytes());

        System.out.println("Original Message: " + message);

        // Encrypt
        BigInteger encrypted = rsa.encrypt(msg);
        System.out.println("Encrypted Message: " + encrypted);

        // Decrypt
        BigInteger decrypted = rsa.decrypt(encrypted);
        String finalMessage = new String(decrypted.toByteArray());

        System.out.println("Decrypted Message: " + finalMessage);
    }
}
```

Pr 7: Implement Digital signature algorithm in Java
Code:

```
import java.security.*;

import java.util.Base64;


public class DigitalSignatureExample {


  // Generate Key Pair (Public + Private)

  public static KeyPair generateKeys() throws Exception {

    KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");

    keyGen.initialize(2048);

    return keyGen.generateKeyPair();

  }


  // Sign the message using Private Key

  public static byte[] signMessage(PrivateKey privateKey, String message) throws Exception {

    Signature signature = Signature.getInstance("SHA256withRSA");

    signature.initSign(privateKey);

    signature.update(message.getBytes());

    return signature.sign();

  }


  // Verify the signature using Public Key

  public static boolean verifySignature(PublicKey publicKey, String message, byte[]
signatureBytes) throws Exception {

    Signature signature = Signature.getInstance("SHA256withRSA");

    signature.initVerify(publicKey);

    signature.update(message.getBytes());

    return signature.verify(signatureBytes);

  }


  public static void main(String[] args) {
```

```
    try {
        System.out.println("---- Digital Signature Demo ----");

        // Step 1: Generate keys
        KeyPair keyPair = generateKeys();
        PrivateKey privateKey = keyPair.getPrivate();
        PublicKey publicKey = keyPair.getPublic();

        // Step 2: Original Message
        String message = "Payment approved: ₹5000";
        System.out.println("Original Message: " + message);

        // Step 3: Sign the message
        byte[] signatureBytes = signMessage(privateKey, message);
        String signatureBase64 = Base64.getEncoder().encodeToString(signatureBytes);

        System.out.println("\nDigital Signature (Base64):");
        System.out.println(signatureBase64.substring(0, 50) + "...");

        // Step 4: Verify (correct message)
        boolean isValid = verifySignature(publicKey, message, signatureBytes);
        System.out.println("\nVerification (Original Message): " + isValid);

        // Step 5: Tampered message
        String tamperedMessage = "Payment approved: ₹50000";
        boolean isTamperedValid = verifySignature(publicKey, tamperedMessage,
signatureBytes);

        System.out.println("\nTampered Message: " + tamperedMessage);
        System.out.println("Verification (Tampered): " + isTamperedValid);
```

```
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Pr 8: Write a program to implement Diffie–Hellman Key Exchange.

Code: import java.math.BigInteger;

```
import java.security.SecureRandom;

public class DiffieHellmanExample {

  public static void main(String[] args) {

    SecureRandom random = new SecureRandom();
    int bitLength = 512;

    // Step 1: Public values (shared)
    BigInteger p = BigInteger.probablePrime(bitLength, random); // prime
    BigInteger g = BigInteger.valueOf(5); // base

    System.out.println("Public Prime (p): " + p.toString().substring(0, 20) + "...");
    System.out.println("Public Base (g): " + g);

    // --- Alice side ---
    BigInteger a = new BigInteger(bitLength - 1, random); // private key
    BigInteger A = g.modPow(a, p); // public key

    // --- Bob side ---
    BigInteger b = new BigInteger(bitLength - 1, random); // private key
    BigInteger B = g.modPow(b, p); // public key

    System.out.println("\nAlice sends A: " + A.toString().substring(0, 20) + "...");
    System.out.println("Bob sends B: " + B.toString().substring(0, 20) + "...");

    // Step 3: Shared secret calculation
    BigInteger sharedAlice = B.modPow(a, p);
```

```
        BigInteger sharedBob = A.modPow(b, p);


        System.out.println("\nShared Secret (Alice): " + sharedAlice.toString().substring(0, 20) +
"...");
        System.out.println("Shared Secret (Bob): " + sharedBob.toString().substring(0, 20) + "...");


        // Verify
        System.out.println("\nDo both secrets match? " + sharedAlice.equals(sharedBob));
    }
}
```

Pr 9: Implementation of SQL Injection Attack
Code:

```
import java.sql.*;

import java.util.Scanner;


public class SQLInjectionDemo {


    public static void main(String[] args) {


        Scanner sc = new Scanner(System.in);


        System.out.println("---- Login System ----");


        System.out.print("Enter Username: ");

        String username = sc.nextLine();


        System.out.print("Enter Password: ");

        String password = sc.nextLine();


        // ❌ Vulnerable query (SQL Injection possible)

        String query = "SELECT * FROM users WHERE username = '"

                + username + "' AND password = '"

                + password + "'";


        System.out.println("\nGenerated Query:");

        System.out.println(query);


        System.out.println("\n ⚠️ If user enters: ' OR '1'='1");

        System.out.println("Login will bypass authentication!");


        sc.close();
```

```
    }
}
```

Prevention code :

```
String query = "SELECT * FROM users WHERE username = ? AND password = ?";
PreparedStatement ps = conn.prepareStatement(query);

ps.setString(1, username);
ps.setString(2, password);
```


Here are **Viva Questions + Simple Answers** for **Practical 1, 2, 3** in a clean, easy-to-explain format (perfect for external exam).

🧪 **PRACTICAL 1: Caesar Cipher**

❓ **1. What is Caesar Cipher?**

👉 It is a simple encryption technique where each letter is shifted by a fixed number (key).

❓ **2. How does it work?**

👉 Each letter in the message is replaced by another letter after shifting it forward by a fixed value.

Example:
HELLO + key(3) → KHOOR

❓ **3. What is the role of the key?**

👉 The key decides how many positions letters are shifted.

❓ **4. How do you decrypt?**

👉 Shift letters backward using the same key.

❓ **5. Can it encrypt numbers or symbols?**

👉 No, it usually works only on alphabets.

❓ **6. Is Caesar Cipher secure?**

👉 No, it is very weak and can be easily broken by trying all keys.

❓ **7. What type of cipher is it?**

👉 It is a **substitution cipher** (letters are replaced).

❓ **8. What is brute force attack?**

👉 Trying all possible keys (1–25) to find the correct message.

❓ **9. What is its advantage?**

👉 Very simple and easy to implement.

❓ **10. Main disadvantage?**

👉 Very low security.

🧪 **PRACTICAL 2: Playfair Cipher**

❓ **1. What is Playfair Cipher?**

👉 It is an encryption technique that works on **pairs of letters (digraphs)**.

❓ **2. How is it different from Caesar Cipher?**

👉 Caesar uses single letters, Playfair uses pairs → more secure.

❓ **3. What is a key matrix?**

👉 A 5×5 matrix created using a keyword, used for encryption.

❓ **4. Why are I and J treated the same?**

👉 Because matrix has only 25 spaces, so one letter (J) is removed.

❓ **5. How are repeated letters handled?**

👉 If two same letters come together, we insert 'X' between them.

Example: BALLOON → BA LX LO ON

❓ **6. What if message length is odd?**

👉 Add 'X' at the end.

❓ **7. What are encryption rules?**

👉 Same row → move right
👉 Same column → move down
👉 Rectangle → swap columns

❓ **8. Is it secure?**

👉 More secure than Caesar but still breakable.

❓ **9. Can it encrypt numbers?**

👉 No, only alphabets.

❓ **10. Advantage?**

👉 Harder to break than simple substitution.

🧪 **PRACTICAL 3: Rail Fence Cipher**

❓ **1. What is Rail Fence Cipher?**

👉 It is a **transposition cipher** that arranges text in zigzag form.

❓ **2. What is meant by zigzag pattern?**

👉 Writing letters diagonally up and down across rows.

❓ **3. How does encryption work?**

👉 Write message in zigzag → read row by row.

❓ **4. How does decryption work?**

👉 Recreate zigzag pattern → fill letters → read original.

❓ **5. What is the key?**

👉 Number of rails (rows).

❓ **6. What happens if rails = 1?**

👉 No encryption happens.

❓ 7. Is it substitution or transposition?

👉 Transposition (positions change, not letters).

❓ 8. Is it secure?

👉 No, pattern can be easily guessed.

❓ 9. Advantage?

👉 Easy and fast to implement.

❓ 10. Disadvantage?

👉 Weak security and no real key protection.

🎯 Final Quick Comparison (Very Useful for Viva)

Cipher	Type	Works On	Security
Caesar	Substitution	Single letter	Very Low
Playfair	Substitution	Letter pairs	Medium
Rail Fence	Transposition	Positions	Low

🎤 How to Answer Smartly in Viva

If examiner asks:
👉 "Explain simply"

Say:

- Caesar → shift letters
- Playfair → encrypt in pairs using matrix
- Rail Fence → rearrange letters in zigzag

If you want next:

- Viva for **4,5,6**
- 1-page **last minute revision sheet**
- Important **expected questions**

Just tell 👍

vivia for 4,5,6

Here are **Viva Questions + Simple Answers** for **Practical 4, 5, 6** in a clean, easy-to-explain format (perfect for external exam).

🧪 **PRACTICAL 4: Columnar Transposition Cipher**

❓ **1. What is Columnar Transposition Cipher?**

👉 It is a method where letters are rearranged in columns using a key.

❓ **2. How does it work?**

👉 Write message in rows → rearrange columns based on key → read column-wise.

❓ **3. What is the role of the key?**

👉 The key decides the order in which columns are read.

❓ **4. What is meant by permutation of columns?**

👉 Changing the order of columns based on alphabetical order of key letters.

❓ **5. What is padding?**

👉 Adding extra characters (like X) to fill empty spaces.

❓ **6. Is it substitution or transposition?**

👉 Transposition (only position changes).

❓ **7. How is decryption done?**

👉 Rearrange columns back to original order → read row-wise.

❓ **8. Is it secure?**

👉 More secure than Rail Fence but still breakable.

❓ **9. Advantage?**

👉 Uses key, so slightly stronger than simple ciphers.

❓ **10. Disadvantage?**

👉 Can be broken using pattern analysis.

🧪 **PRACTICAL 5: One-Time Pad (OTP)**

❓ **1. What is One-Time Pad?**

👉 It is a symmetric encryption method using a random key of same length as message.

❓ **2. Why is it called "One-Time"?**

👉 Because the key is used only once.

❓ **3. How does it work?**

👉 Each letter of message is combined with key using mathematical operation (mod 26).

❓ **4. What is the main requirement of OTP?**

👉 Key must be:
✓ Same length
✓ Random
✓ Used only once

❓ **5. Why is OTP unbreakable?**

👉 Because the key is random and has no pattern.

❓ **6. What operation is used?**

👉 Modular arithmetic (addition and subtraction).

❓ **7. What happens if key is reused?**

👉 It becomes insecure and can be broken.

❓ **8. Is OTP practical?**

👉 Not much, because key management is difficult.

❓ **9. Advantage?**

👉 Perfect security (theoretically unbreakable).

❓ **10. Disadvantage?**

👉 Difficult to generate and share large random keys.

🧪 **PRACTICAL 6: RSA Algorithm**

❓ **1. What is RSA?**

👉 RSA is an **asymmetric encryption algorithm** using two keys.

❓ **2. What are the two keys?**

👉 Public Key (for encryption)
👉 Private Key (for decryption)

❓ **3. How are keys generated?**

👉 Using two large prime numbers.

❓ **4. What is the role of public key?**

👉 Used to encrypt the message.

❓ **5. What is the role of private key?**

👉 Used to decrypt the message.

❓ **6. Why is RSA secure?**

👉 Because factoring large prime numbers is very difficult.

❓ 7. What type of encryption is RSA?

👉 Asymmetric encryption.

❓ 8. What is modulus (n)?

👉 Product of two prime numbers (p × q).

❓ 9. Where is RSA used?

👉 HTTPS, banking, secure communication.

❓ 10. Advantage?

👉 High security and widely used.

❓ 11. Disadvantage?

👉 Slower compared to symmetric encryption.

🎯 Final Quick Comparison (Important)

Cipher	Type	Key Type	Security
Columnar	Transposition	Single key	Medium
One-Time Pad	Substitution	Random key	Very High
RSA	Asymmetric	Public + Private	Very High

🎤 Smart Viva Answer Tip

If examiner asks:
👉 "Explain in simple words"

Say:

- Columnar → rearranges letters using a key
- OTP → uses random key, very secure
- RSA → uses two keys for encryption & decryption

If you want next:

- Viva for **7,8,9**
- Full **important questions list**
- Last-minute **revision sheet**

Just tell 👍

vivia for 7,8,9

Here are **Viva Questions + Simple Answers** for **Practical 7, 8, 9** in a clean, easy-to-explain format (perfect for external exam).

🧪 **PRACTICAL 7: Digital Signature**

❓ **1. What is a Digital Signature?**

👉 It is a method to verify that a message is **authentic and not changed**.

❓ **2. What are the main purposes?**

👉
✓ Authentication (who sent it)
✓ Integrity (not modified)
✓ Non-repudiation (cannot deny sending)

❓ **3. How does it work?**

👉 Message → Hash → Encrypt with private key → Signature

❓ **4. How is verification done?**

👉 Decrypt signature using public key → compare with hash of message

---

❓ **5. Which keys are used?**

👉 Private key → signing
👉 Public key → verification

❓ **6. What is hashing?**

👉 Converting message into a fixed-size value (digest).

❓ **7. Why is hashing used?**

👉 To detect any change in message quickly.

❓ **8. What happens if message is changed?**

👉 Signature verification fails.

❓ **9. Is it same as encryption?**

👉 No, it is used for verification, not secrecy.

❓ **10. Where is it used?**

👉 Online transactions, emails, contracts

🧪 **PRACTICAL 8: Diffie–Hellman Key Exchange**

❓ **1. What is Diffie–Hellman?**

👉 A method to **securely exchange keys over an insecure network**.

❓ **2. What is its main purpose?**

👉 To generate a shared secret key between two users.

❓ **3. What are public values?**

👉 Prime number (p) and base (g)

❓ **4. What are private keys?**

👉 Secret numbers chosen by users (a and b)

❓ **5. How does it work?**

👉

A = g^a mod p
B = g^b mod p
Shared key = same on both sides

❓ **6. Why is it secure?**

👉 Because it is hard to calculate private keys from public values.

❓ **7. What problem does it solve?**

👉 Secure key exchange without sending key directly.

❓ **8. What is its weakness?**

👉 No authentication → vulnerable to man-in-the-middle attack.

❓ **9. Is it encryption?**

👉 No, it is only for key exchange.

❓ **10. Where is it used?**

👉 HTTPS, VPN, secure communication

🧪 **PRACTICAL 9: SQL Injection Attack**

❓ **1. What is SQL Injection?**

👉 A type of attack where malicious SQL code is inserted into input fields.

❓ **2. How does it work?**

👉 User input is directly added into SQL query → attacker modifies query.

❓ **3. Give a simple example**

👉
Input: ' OR '1'='1
👉 Makes condition always true → login bypass

❓ **4. What is the main vulnerability?**

👉 Using string concatenation in SQL queries.

❓ 5. What are the impacts?

👉
✓ Data theft
✓ Unauthorized access
✓ Data deletion

❓ 6. What is authentication bypass?

👉 Logging in without correct username/password.

❓ 7. How to prevent SQL Injection?

👉
✓ Prepared Statements
✓ Input validation
✓ Parameterized queries

❓ 8. What is prepared statement?

👉 A secure way to execute SQL queries without mixing input directly.

❓ 9. Is SQL Injection dangerous?

👉 Yes, it can compromise entire database.

❓ 10. Where does it occur?

👉 Web applications, login forms, search fields

🎯 Final Quick Comparison (Important)

Practical	Concept	Purpose
Digital Signature	Authentication	Verify message
Diffie–Hellman	Key Exchange	Share secret key
SQL Injection	Attack	Exploit database

🎤 **Smart Viva Answer Tip**

If examiner asks:
👉 “Explain simply”

Say:

- Digital Signature → verifies message authenticity
- Diffie-Hellman → shares secret key securely
- SQL Injection → attack using input manipulation

---

give me first practical code with humanised version in java and explain what it do in simple language

Here is your **1st practical (Caesar Cipher)** rewritten in a clean, human-friendly Java version, along with a simple explanation.

✅ Humanized Java Code (Caesar Cipher)

```
public class CaesarCipher {

  // Method to encrypt text
  public static String encrypt(String text, int key) {
    StringBuilder result = new StringBuilder();

    // Normalize key (handles keys > 26)
    int shift = key % 26;

    for (int i = 0; i < text.length(); i++) {
      char ch = text.charAt(i);

      // For uppercase letters
      if (Character.isUpperCase(ch)) {
        char encryptedChar = (char) ('A' + (ch - 'A' + shift) % 26);
        result.append(encryptedChar);
      }
      // For lowercase letters
      else if (Character.isLowerCase(ch)) {
        char encryptedChar = (char) ('a' + (ch - 'a' + shift) % 26);
        result.append(encryptedChar);
      }
      // For spaces or symbols (unchanged)
      else {
        result.append(ch);
      }
    }

    return result.toString();
  }

  // Method to decrypt text
  public static String decrypt(String text, int key) {
    int shift = 26 - (key % 26); // reverse shift
    return encrypt(text, shift);
  }

  public static void main(String[] args) {
```

```
        String originalText = "Hello World";
        int key = 3;

        System.out.println("Original Text: " + originalText);

        String encrypted = encrypt(originalText, key);
        System.out.println("Encrypted Text: " + encrypted);

        String decrypted = decrypt(encrypted, key);
        System.out.println("Decrypted Text: " + decrypted);
    }
}
```

🧠 Simple Explanation (Very Easy Language)

This program is based on **Caesar Cipher**, which is one of the simplest encryption methods.

🔐 What it does:

- It **hides a message** by shifting each letter forward.
- Example:
  - If key = 3
  - A → D
  - B → E
  - HELLO → KHOOR

⚙️ How the code works:

1. Encryption

- Takes text + key
- Moves each letter forward by the key value
- Keeps spaces/symbols unchanged

2. Decryption

- Does the opposite (moves letters backward)
- Uses formula:

26 - key

- Calls same encrypt function (smart reuse)

🔁 Example Output:

Original Text: Hello World
Encrypted Text: Khoor Zruog
Decrypted Text: Hello World

💡 **Key Points**

- Works only on alphabets (A–Z, a–z)
- Very easy to understand and implement
- Not secure for real-world use (can be cracked easily)

🎯 **In One Line**

👉 This program shifts letters to hide the message and shifts them back to get the original text.

If you want, I can also:

- Convert this into **menu-driven code (1: Encrypt / 2: Decrypt)**
- Or give **viva answers + theory in simple words**